Mattias Hansen, Alessandro Caldara

PROJET TRANSVERSE I: CAHIER DES CHARGES

Introduction et Contexte

Nous avons sélectionné le sujet n° 2 : Messagerie instantanée car ce sujet présente un grand intérêt pour nous. Il nécessitera l'implémentation d'un chiffrement end-to-end, de comptes utilisateurs, de fonctionnalités communes aux messageries populaires (sondages, messages épinglés,...) et d'algorithmes de recherche, ainsi que d'une infrastructure client-serveur à la fois simple et robuste, garantissant des échanges rapides. Après discussion et approbation, nous allons orienter nos efforts vers le développement d'une messagerie instantanée de type Discord, tout en y intégrant certaines spécificités. Nous voudrions idéalement essayer de pousser l'anonymat jusqu'à la non-nécessité d'utiliser un email pour s'identifier, mais plutôt un code de 16 chiffres (à la manière Mullvad VPN). Ce choix est notamment motivé par la récente controverse concernant Discord, qui prévoyait d'introduire une vérification de l'âge par des moyens jugés intrusifs par un grand nombre d'utilisateurs (nous inclus), et qui s'était associé à l'entreprise Persona, connue pour ses liens avec Peter Thiel – ce qui constitue une raison supplémentaire de s'inquiéter pour l'avenir de la protection des données, de l'identité et de la sécurité des internautes.

Objectif du projet

- Le but est de fournir une plateforme permettant à des utilisateurs authentifiés de communiquer en temps réel. Le système permettra:
    - La création d'espaces de discussion (dits “Serveurs”)
    - L'organisation de ces espaces en sous-sections thématiques (dits “Salons” ou “Channels”)
    - L'envoi de messages privés entre deux utilisateurs
    - La recherche de messages par mots-clé
    - La mise en épinglé d’un message par Salon
    - L’envoie de différents types de messages (courts, sondages,...)

Description des exigences (liste - fonctionnelles, non-fonctionnelles)

- FONCTIONNELLES
    - Inscription et création de comptes (avec pseudo, mot de passe, email), sans données biométriques ou lien avec identité réelle
    - Authentification (connexion/déconnexion) sécurisée
    - Gestion du profil utilisateur (modification pseudo, potentielle photo de profil, mot de passe,...)
    - Création d’un serveur par un utilisateur (qui devient Owner/Propriétaire)
    - Génération de code d’invitation pour rejoindre un serveur en tant que simple membre
    - Création, modification et suppression de salons textuels par le ou les utilisateur(s) avec les permissions administrateur

    - Gestion des permissions basiques (ici, nous aurons seulement 2 modes: admin et normal) comme l'expulsion de membres, suppression de messages, mis en épinglé d'un message pour un salon
    - Envoi et réception de messages textuels en temps réel dans les salons
    - Envoi et réception de messages textuels en temps réel entre un utilisateur et un autre
    - Modification et suppression de ses propres messages
    - Création et participation à des messages type "sondage"
    - Épinglage de messages dans un salon (inclut ajout et retrait)
    - Recherche de messages par mots-clés au sein d'un salon ou d'une conversation privée
- NON-FONCTIONNELLES
    - Chiffrement de bout en bout des messages privés (peut être aussi des messages en salon publiques) → stockage des messages sous leur forme de ciphertext
    - Le serveur ne doit stocker les mots de passes (peut être même aussi les identifiants) que sous leurs versions hachés et salées
    - Le serveur ne doit stocker aucune métadonnée non-essentielle au routage des messages
    - Les communications ne doivent pas dépasser un certain seuil de latence (par exemple 200ms)
    - Les messages ne doivent pas être confinés à une "session" et doivent être accessibles même par un utilisateur plus "jeune" (en terme de date de création ou d'accès) qu'un message lui-même
    - L'Interface doit être facile à naviguer (et peut être conçue comme familière à un utilisateur de Discord)
    - L'interface doit pouvoir prendre en charge d'un "mode sombre"

Description générale/Architecture fonctionnelle

- Front-end
    - Fournit l'environnement visuel de l'utilisateur (incluant le mode sombre), structuré autour de la navigation entre les serveurs, les salons et les messages privés.
    - Gère localement la génération des clés, le chiffrement des messages avant leur émission et leur déchiffrement à la réception.
    - Maintient une connexion continue avec le serveur (par exemple via des WebSockets) pour assurer la faible latence des échanges. Puisque les messages sont chiffrés, l'algorithme de recherche s'exécute localement sur le client.
- Back-end
    - Développé en Python (avec Flask), le serveur gère la création des comptes, l'authentification et s'assure du respect des permissions (vérification des droits "admin" pour la suppression ou l'épinglage de messages).
    - Serveur réceptionne et stocke les messages sous forme de ciphertext et se contente de les redistribuer instantanément aux clients connectés (dans un salon ou en privé), sans pouvoir en lire le contenu.
- Base de données

    - Permet la persistance de l'architecture des serveurs et des salons pour que l'historique soit accessible de manière asynchrone (même pour les nouveaux membres).
    - Les mots de passe y sont stockés uniquement sous forme hachée et salée. Les messages y sont sauvegardés chiffrés, avec le strict minimum de métadonnées requises pour leur affichage (identifiants, horodatage, statut de l'épingle)

Parties prenantes (utilisateurs)

- Utilisateur standard**:** Nous voulons qu'un utilisateur normal puisse utiliser notre app pour communiquer tous les jours sans se soucier que quelqu'un puisse récolter des données personnelles sur lui. En principe, il ne devrait pas avoir des permissions spéciales, et n'a que le strict minimum comme écrire et lire des messages dans des salons.
- Administrateur**:** En plus des attentes d'un membre standard, il a besoin d'outils simples et fiables pour structurer son "serveur" (création et gestion des salons) et la modérer efficacement (génération de codes d'invitation, expulsion de membres, suppression ou épinglage de messages). Chaque utilisateur est en principe considéré comme Administrateur dans les messages privés (en 1-à-1)
- Propriétaire: C'est un administrateur mais il s'agit de celui qui a fondé le serveur et il possède toutes les permissions, inclues celles qui l'autorisent à appointer ou exclure des administrateurs.

Livrables attendus

- Rapport: document présentant la solution finale et complète, en expliquant choix architecturaux, difficultés rencontrées et améliorations futures possibles.
- Présentation: ne doit pas dépasser les 20 minutes. Il faut utiliser des transparents comme support.
- Code source du Front End et Back End (git + conteneurisation)
- Prototype fonctionnel sur machine locale et/ou serveur de test qui inclut le minimum: authentification, création de salon et envoi/réception de messages chiffrés
- README qui explique clairement comment faire fonctionner l'application.